ST446 WT2026 - Assessment 2 Part 1
Detecting Fake Political Statements with PySpark MLlib
Candidate Number: 61396

Problem Definition & Motivation

Introduction

It is easy in the modern day to spread false statements and for people to interpret the statements as true. Media sites, tv airings of speeches, interviews, emails and other ways of spreading news are a part of our daily lives. Politicians and figures have extreme power in manipulating what is considered true and what is considered fake news. A system of monitoring disinformation that can handle the massive amounts of digital data and digital methods of spreading statements from a politician is crucial to prevent harm to citizens and corruption by politicians. Therefore, my research question is: how can machine learning detect fake news in political statements that are accessed and stored digitally? Specifically, I will be investigating data that originates from PolitiFact's online records. PolitiFact is a "fact-checking website that rates the accuracy of claims by elected officials and others" (PolitiFact).

The dataset I utilize in my investigation is the "LIAR" dataset, created by William Wang in 2017. This dataset compiles data from over a decade of 12,800 manually labeled short statements in various contexts from PolitiFact.com. The original labels for categories of truthfulness are: true, mostly true, half true, barely true, false, and pants on fire (extremely false). The other variables included are the id of the statement JSON file, text of the statement, subject of the statement, speaker's name, speaker's job title, state, speaker's party affiliation, location or online place the statement was made, and the speaker's total credit history count (barely true, false, half true, mostly true pants on fire counts). The target variable I will use is a binary label where the original labels of true, mostly-true and half-true are considered true and the others as false. I was more interested in a binary target rather than the 6 different categories as the applications could be more useful for broadly detecting fake statements.

In a big data context, this problem is relevant as it is easier than ever for misinformation to spread online with social media apps, news sites, videos and more. The amounts of data related to political statements are massive and no one human can go through each and every one to check for misinformation. The rate at which politicians make statements that are spread around and analyzed online is too quick to fully address without a machine. Additionally, human labeling could be biased if the person determining if a statement is true or false has a political preference. Detecting false news is harder than ever when a statement can be determined to be fake by one group of people and true by another. Thus, methods to handle and model big data are needed to solve this issue of detecting false political statements.

Data Processing & Feature Engineering

Data Cleaning

As mentioned previously, my goal is to build a model that can label a statement as false or true. I selected the 'LIAR' dataset for this and loaded it in from the publicly available GitHub repository "liar_dataset" from the user "tfs4". The user originally stored the files as tab-separated values files split into training, testing and validation sets. Instead of using this split, I loaded in the .tsv files and combined them into one full dataset. I set my own schema where id, label, statement, subject, speaker, subject_job, state, context, and party were StringType and barely_true_count, false_count, half_true_count, mostly_true_count and pants_on_fire_count were DoubleType. After defining this schema, I loaded and unionized the three original .tsv files into a Spark DataFrame.

Once I had my full DataFrame, I converted the original "label" column to be binary where 0 represents false and 1 represents true based on the criteria mentioned previously. I used the "with column" function to replace the original label entries with the binary label.

Next, I investigated if any missing values were present. I first counted the Na, NaN and Null values and found 3580 for speaker_job, 2757 for state, 137 for context and 8 for all other columns excluding id, label and statement which had 0 missing values. Since there were a lot of missing values for the categorical variables speaker_job, state and context, I decided to, instead of dropping the rows, fill the missing values with "UNKNOWN." The fact the speaker's job title and state are not known could be interesting if the label was more true or false on average. I also dropped the remaining 8 rows with missing values as there was a mix of categorical and numerical values missing and there was not a substantial amount of data lost.

EDA

After handling missing values, I did exploratory data analysis on the other variables in the DataFrame. I first aggregated the rows to count 7163 true labeled and 5665 false labeled rows. This is about 56% of true labels and 44% of false label balance for the data. Aggregating a bit more granularly, we can see the top political groups of "republican" have a pretty even split between false vs. true but democrat has 1335 more true than false labels (Figure 1). This could present possible bias in the model since it trains on more democrats having true labels. It is unclear if this is PolitiFact labeling more democrats as true or if democrats in reality have more true than false statements. In further studies I could test my model on data from other sources than PolitiFact.

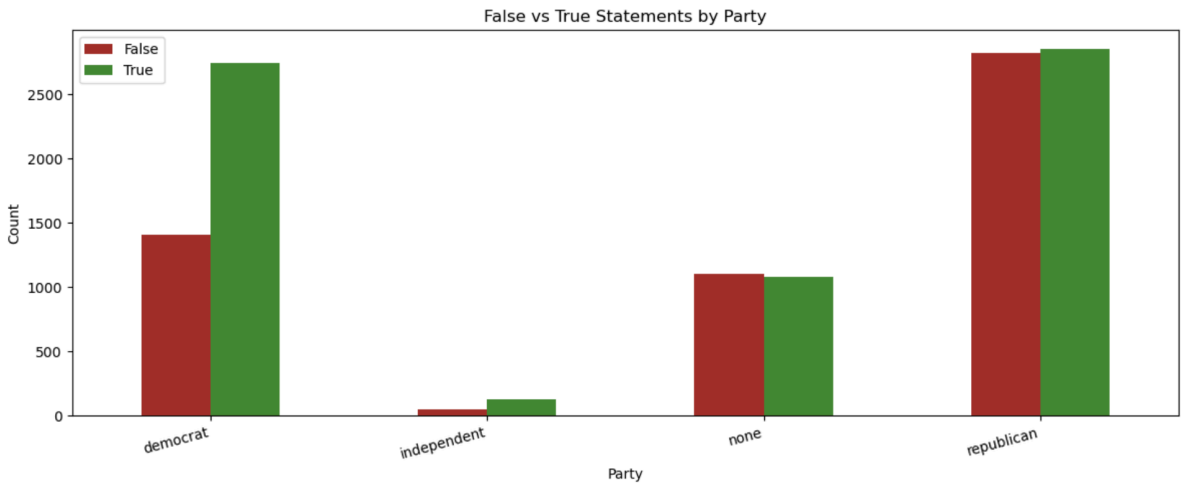

Figure 1: False vs. True Statement Counts By Party

Next, I looked at the 'state' counts by unique values. It was revealed there were many entry issues for this column that I needed to address. First, some states had trailing white spaces, causing them to register as a different unique value than the original state. I removed this with the trim function in the with column function. There were also spelling and capitalization issues in entries like in "Tex" or "ohio" which I matched to the intended states. Similarly, Washington D.C. had multiple different variants which I matched. Lastly, I filled the non-US state entries (like "China" and "Qatar") with "UNKNOWN" as the politicians and statements in the dataset were primarily U.S. related. I also investigated the percentage of fake statements, queries by state (Figure 2). Minnesota had 70% of fake statements and other states like Alaska, Tennessee, Pennsylvania and Indiana had over 50%. Again, one could look into if PolitiFact has bias against Minnesota and other states or if this high proportion of false labels does not affect results when tested on other data sets.

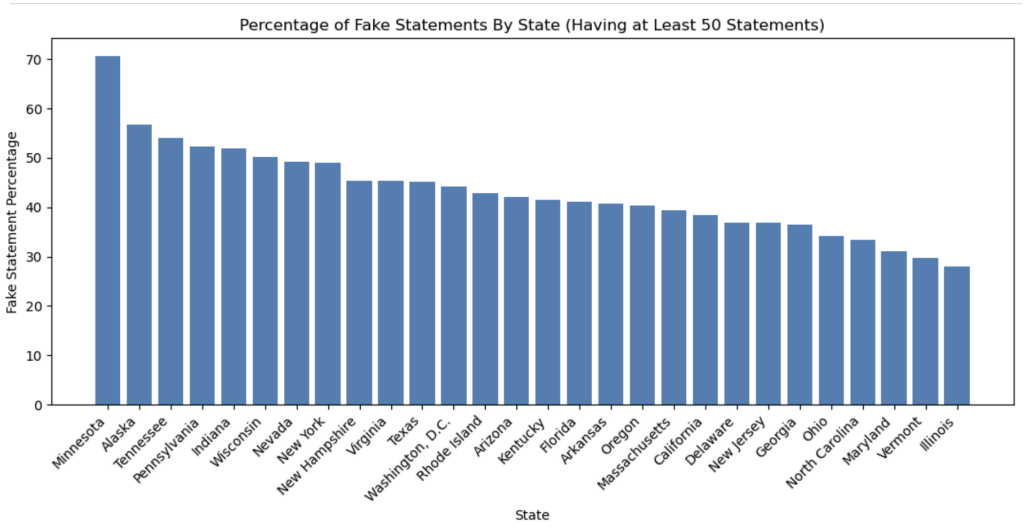

Figure 2: Percent of Fake Statements by State

Using the explode and splitting functions for text analysis, I extracted the most common words in the “Subject” and “Context” columns. Below are the top 10 in count for each (Table 1 and Table 2). We can see the top subjects are financial topics like economy, taxes, federal and state budgets. The forms of how the news was shared is pretty diverse with categories like news release, interviews and social media posts.

Subject	Count
economy	1435
health-care	1432
taxes	1220
federal-budget	943
education	930
jobs	902
state-budget	884
candidates-biography	808
elections	758
immigration	645

Table 1: Top 10 Statement Subjects

Media Form	Count
a news release	311
an interview	286
a press release	282
a speech	262
a TV ad	225
a tweet	190
a campaign ad	164
a television ad	163
a radio interview	127
a debate	114

Table 2: Top 10 Media Forms

Table 3 shows the top 6 fake and real labeled statements by the speaker. Obama had 26.2% false label rate, Trump had 70.3%, Hillary Clinton had 26.6% and Romney, McCain and Walker had around 41-45%. It is worth noting the data is from 2007-2017 where Obama was the president from 2009-2017 and Hillary Clinton, Romney and McCain all had campaigns during these years. This could explain the high counts of statements from them in the dataset. While Trump was not campaigning all of these years, he was active on social media and in the news for criticizing Obama for allegedly not being born as a U.S citizen. He was actively spreading verified false news during this time which could explain the high percentage of false labels connected to Donald Trump in the dataset. Figure 3 shows the counts for the top political job titles. “President” and “Presidential Candidate” have more true than false labels which could connect back to Obama being president at the time where we saw the 26.2% false label. “President-Elect” has more false than true, also relating to Donald Trump being elected in 2016. In summary, these imbalances in false and true labels for certain variables like party, candidates and job titles could affect a model's bias or performance.

Speaker	Party	Fake	Real	Total
Barack Obama	Democrat	161	453	614
Donald Trump	Republican	242	102	344
Hillary Clinton	Democrat	79	218	297
Mitt Romney	Republican	89	126	215
John McCain	Republican	80	109	189
Scott Walker	Republican	84	100	184

Table 3: Fake and Real Statement Counts by Speaker

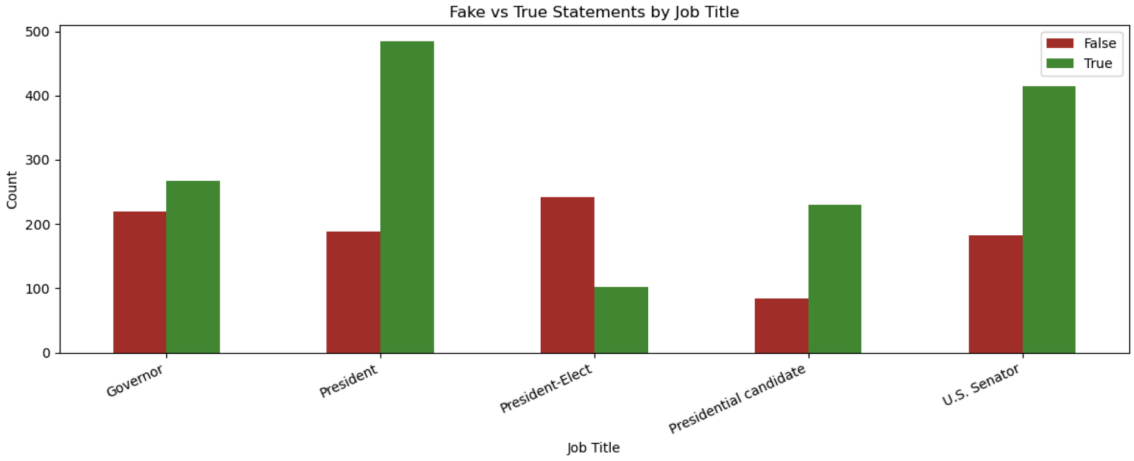

Figure 3: Fake vs. True Statement Counts by Top Job Titles

Feature Extraction and Transformation

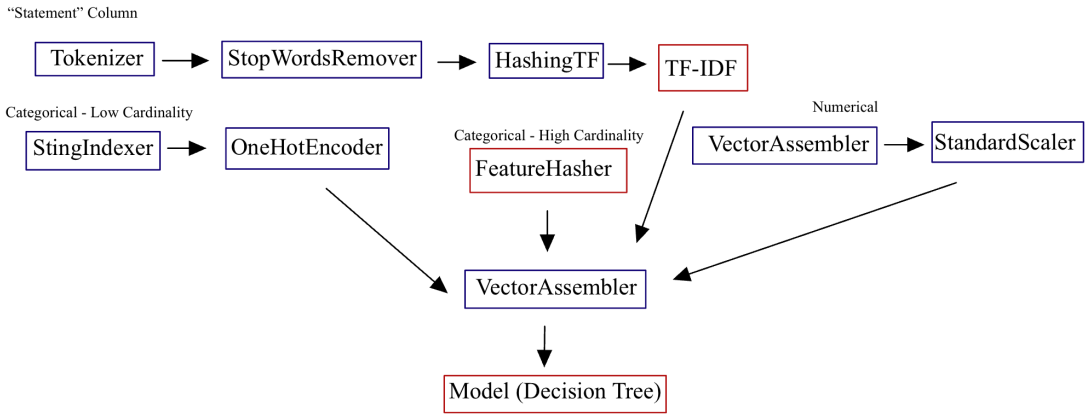

Figure 4: Pipeline. Transformers are in navy boxes, Estimators are in red boxes.

For extracting and transforming features, I used the Spark MLlib pipeline shown in Figure 4. To preprocess the text in "statement," I extracted individual words using the Tokenizer transformer. Each statement string is split on each whitespace, resulting in individual words being stored as tokens. This is needed as the following parts of the pipeline utilize "tokens" and not the full raw strings. The next transformation of the individual words or tokens is through StopWordsRemover where "stop words" or uninformative and common words like "the" and "is" are removed. This helps reduce noise and helps the models learn meaningful words in the statement. The next transformation is in HashingTF where the word or token frequencies are calculated using the "hashing trick" (Apache Software Foundation). HashingTF maps the sequence of words to a sparse vector with a specified dimension of 300. The values in the vector are the frequency that the word or token was hashed. The default for the number of dimensions is 262,144 but due to

limitations of memory and computer efficiency, I reduced it to 300 since the PolitiFact statements are all short (Anthropic, Claude Sonnet 4.6, 2026). HashingTF is useful for scaling and handling big data as no vocabulary dictionary is needed, memory storage is fixed with the dimension size and text can be converted into numerical vectors. The last modification of the statement words uses the feature extractor TF-IDF. Here, the term frequency (TF) comes from the HashingTF resulting vector. The Inverse Document Frequency (IDF) adjusts the frequencies by multiplying the frequency by how rare the word is. The rarer the word, the higher the IDF value. IDF is an Estimator since it fits on the training data to determine what words are considered rare or not.

The categorical variables have different levels of cardinality and are addressed differently. The low cardinality variables were party and state with 24 and 60 unique values respectively. Subject had 4547, context had 5157, speaker had 3317 and speaker_job had 1362 unique values.

For state and party (low cardinality), I used the Transformers StringIndexer and OneHotEncoder. StringIndexer maps the string to numerical indexes. This transformation is needed since the models take in numeric inputs. OneHotEncoder converts the indices to binary sparse vectors. This takes away a possibility that the model registers an ordinal relationship of the StringIndexer indices.

For the high cardinality variables, the Estimator FeatureHasher is used. StringIndexer and OneHotEncoder would not be efficient for the high number of distinct values. There would be too many columns of binary vectors. FeatureHasher uses the hashing trick to project the set of categorical features to a feature vector of a specified dimension (Apache Spark). The default dimension is 262,144 which I had to drastically reduce to 128. While this may have lost the ability to minimize hash collisions, it was needed to prevent memory crashes in my testing. FeatureHasher combines the 4 high cardinality column values and combines them into one string. Then the hashing trick is applied and the position in the feature vector is outputted. A 1.0 is placed in the location (out of 128), essentially being “one-hot” encoded.

The remaining numerical variables (credit history counts) are transformed with VectorAssembler and StandardScaler. VectorAssembler merges the multiple columns into one vector column. StandardScaler then standardizes each feature in the vectors by dividing by the standard deviation of the respective variable (barely_true_count, false_count, half_true_count, mostly_true_count and pants_on_fire_count).

Lastly, the TF-IDF features, one hot encoded state, party, FeatureHasher features, and standardized count history values are merged into one feature vector with VectorAssembler. This is necessary so that the Spark MLlib models can function as they only take in one vector.

Baseline Model

The baseline model is a Decision Tree Classifier from PySpark MLib learn and is incorporated into the pipeline defined previously. Decision trees have logarithmic predicting cost to the number of data points used to train the tree (Scikit Learn). They can also handle both linear and non-linear relationships. The assumptions are that the data can be partitioned with a series of yes or no questions at each node. The tree depth used was 5 along with the rest of the default values for a Spark DecisionTreeClassifier.

With this decision tree as the final step, I fit the defined pipeline on the training dataset so the estimators learn only on the transformed training split of data. Then the test set data is transformed in the same pipeline and the final feature vector is inputted into the fitted decision tree for predictions.

To evaluate the predictions, I defined an evaluation function which calculates the accuracy, AUC-ROC, F1 Score, Weighted Precision and Weighted Recall. It utilizes the evaluators BinaryClassificationEvaluator and MulticlassClassificationEvaluator.
BinaryClassificationEvaluator is used for AUC-ROC as it takes in a raw prediction column to use probabilities as the cutoff changes from above and below 0.5.
MulticlassClassificationEvaluator is used for the other metrics.

The metrics I use for evaluation are:

- **Accuracy**: The total true positives and true negatives divided by the total predictions. This is the overall correctness of predicting. If the classes are imbalanced, this value can be misleading if it predicts the majority class often and is not effective with the minority class. I include this in my evaluation to give a general overview of model performance.
- **AUC-ROC**: The Area Under the Receiver Operating Characteristic Curve. The ROC curve is the true positive rate (identified true correctly) vs. the false positive rates (false statements labeled as true) over all classification thresholds (not just above or below 0.5). AUC-ROC is the area under this curve and the closer the area is to 1, the better the model correctly identifies true statements and avoids labeling false statements as true. I include this in my evaluation to measure how well the model separates the two classes among all thresholds. It is more robust than accuracy and well suited if classes are imbalanced.
- **Weighted Precision**: The proportion of statements that were correctly labeled. For each class (false and true), precision is calculated by taking the correctly labeled statements in a class divided by the total predications made for a class. The final metric is the weighted average based on total samples for each class. This will show how reliable the model is.
- **Weighted Recall**: The proportion of actual false and true statements that the model found and correctly identified. For each class (false and true), recall is calculated by taking the proportion of statements labeled as their actual value out of all actual values in the class. This will show how well the model can find the actual true or false labels.

- **F1 Score**: F1 Score is the average of weighted precision and weighted recall. It is good for unbalanced class size in the dataset. I use this as my primary evaluator since it is more informative than accuracy in potentially imbalanced datasets and balances out performance in recall and precision.

The results for my baseline model were Accuracy = 0.69619, AUC-ROC = 0.62116, F1 = 0.68033, Precision = 0.70469, Recall = 0.69619. The AUC-ROC is a bit lower than the other metrics, signaling a confidence issue when distinguishing between false and real statements over all classification thresholds. The true positive and false positive rates are not well separated. The higher accuracy, precision and recall are possibly due to class imbalances identified in the EDA section. The base model used a shallow tree, leading to potential high bias and underfitting since the splits are not complex enough. Improvements on this model will be trialed with tuning the hyperparameters of the base Decision Tree and bagging.

Advanced Models

Sophisticated Model - Hyper-parameter Tuning

My first sophisticated model incorporated hyper-parameter tuning on the base model. The pipeline is the same as before. The evaluator was the accuracy for the cross validation. The grid search using ParamGridBuilder was max depth: [10,15,20], minimum instances per mode: [5,10,15] and impurity = gini index or entropy. This grid was inputted into CrossValidator, which also used the pipeline as the estimator, the accuracy evaluator and 3 for number folds. Each hyperparameter was trained on the three data folds and the best pipeline and hyperparameter combination was extracted. Then, this best model and pipeline was tested on the test set and the predictions were generated. The final accuracy and best parameters were: Accuracy = 0.70968, AUC-ROC = 0.73566, F1 = 0.70454, Precision = 0.70846, Recall = 0.70968 and maxDepth = 10, minInstancesPerNode = 15, impurity = gini.

This improved on the base model as all metrics are similar and above 0.70 and the AUC-ROC increased by over 0.1. The sophisticated model can better separate true positive and false positive rates over all classification thresholds. Increasing the depth allowed learning for more complex split rules to lower bias and minimize underfitting. Increasing minInstancesPerNode prevented the deeper tree from overfitting as each node had enough training samples in order to split. The impurity stayed the same as the default in the base model.

Sophisticated Model - Bagging

A second sophisticated model I built was a bagging ensemble of Decision Trees. Bagging consists of training multiple independent classifiers on a bootstrap random subset of the data. The overall prediction is the majority vote of all the classifiers. Bagging helps reduce variance and prevent overfitting.

The weak learners I implemented for bagging were deep Decision Trees with a max depth of 30. I used the same pipeline and data transformation as in previous sections (without the basic or tuned Decision Tree from before). Then, in a bagging function a bootstrap sample with replacement was collected and the weak learner was fit on the subset of data. This process was performed for 30 iterations and the F1 score for each of the 30 models varied between 0.65 and 0.68. For each observation in the test set, the majority vote for all 30 classifiers was taken for aggregation. The resulting metrics of the aggregated model were: Accuracy = 0.70783, AUC-ROC = 0.78721, F1 = 0.70361, Precision = 0.70606 and Recall = 0.70783. These results are similar to the tuned decision tree and show improvement on the base learner. By having each tree see a different random bootstrap sample of the data, the errors varied and could be abated by the majority vote, improving generalization. This shows an improvement over the single decision tree, especially in the AUC-ROC value. The bagging ensemble better separated the false and true classes. Accuracy, precision, recall and F1 score being similar also indicates no alarming overfitting. Overall, the bagging ensemble improved on the base model, reduced variance and kept a low bias.

Evaluation & Discussion

Discussion of Results

Table 4 shows the summary of the 3 models I trained. The justification for calculating these metrics is in the “Baseline Model” section of the report. The original base Decision Tree underfit the data, so it had a higher bias. The AUC-ROC was lower than other metrics, showing a struggle in confidently separating class labels. The tuned tree showed improvement over the base and had similar accuracy and other metrics to bagging. Bagging used deep and high variance trees, but aggregation makes it a strong learner that reduces variance. The AUC-ROC improved greatly over the base model and slightly on the tuned tree. This shows a better ability to separate false and true label classes and that the aggregation helped address issues in the base Decision Tree.

Model	Accuracy	AUC-ROC	F1	Precision	Recall
Base DT	0.69619	0.62116	0.68033	0.70469	0.69619
Tuned DT	0.70968	0.73566	0.70454	0.70846	0.70968
Bagging	0.70783	0.78721	0.70361	0.70606	0.70783

Table 4: Model Evaluation Results

Comparative Scenario - Variable Data Size

For a comparative scenario, I trained the three models on different sizes of the datasets and timed how long it took for training. Figure 5 shows the results of the F1 score and training time as the

data size changed. I chose F1 as the comparison metric since it is informative, even with unbalanced class size. The tuned decision tree performs the best of the three at each size of data but does take more time to train. The F1 score peaks at 75% of the data. The training time increases linearly showing reasonable scalability. The bagging model performs decently with 25% of the full data set and shows a steady increase in F1 score as data size increases. The training time is in between the other learners and grows quite slowly, indicating good scalability. The base learner performs well using 25% of the data but underperforms comparatively at 50% of the data. Then the F1 score improves again with 75% but shows instability. The training time is very small at each data size, but this is expected as it has very simple architecture. Further studies could run this comparison on larger datasets than the LIAR original size to see how the tuned and bagging models continue to compare in F1 score.

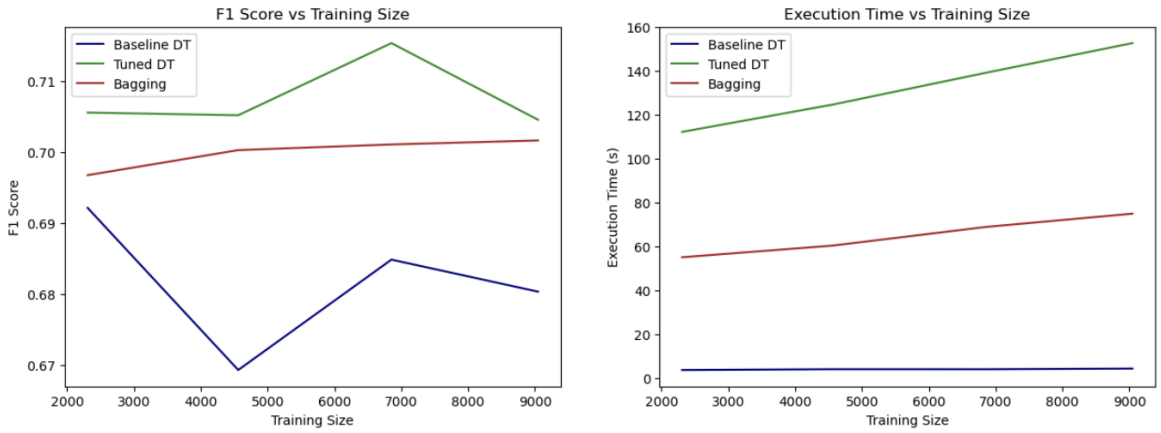

Figure 5: Comparison of F1 Score and Training Time with Dataset Size

Conclusion and Limitations

My investigation into if machine learning can detect true or false statements included training a base Decision Tree, tuning this base Decision Tree and comparing it to a bagging ensemble. The tuned and bagging models improved over the base learner. The bagging showed the best results, especially in AUC-ROC, indicating this model could be the best candidate of the three for separating what is true and what is false. My final comparison study showed a linear increase in training time for the models to investigate scalability of the pipeline. In the real world, there are many more resources and places to gather data on politicians making statements. Further studies would need to test on other datasets and continue studying how the models perform on much larger datasets.

The limitations relate to potential bias in the dataset and the memory capacity when creating the pipeline. Due to limited memory space on my Dataproc cluster, I could not fully increase hashing features to prevent hashing collisions on the high cardinality variables. I also collapsed the

original six labels into binary labels. Some of the categories I put into true or false were not 100% true or 100% false. The models trained on labeling statements that are not fully true or false as true or false. A further study could output the percentage of truth as a target rather than a binary label. Lastly, the dataset itself has bias. All statements were labeled by PolitiFact and came from one time period where certain political figures in office took up a majority of the news talk. The models I trained then might perform poorly on statements made in other eras. Additionally, there was an imbalance in the labeling for democrats and republicans which would need further investigation. An outside source from PolitiFact could review if their labels are consistent with other experts or show any bias. Another option is to train the models on many different sources so any bias with this one source could be averaged out.

GenAI Use

I used GenAI for guidance in four instances. My first prompt was: “How does lowering numFeatures in HashingTF affect modeling in PySpark? Give a short answer.” I was unsure if I could decrease the features in HashingTF to stop my notebook from crashing due to memory storage and still have decent features to work with. It answered that decreasing leads to hash collision increase, speed improvement, risk of underfitting and accuracy changes. I used this to inform my decision to keep HashingTF but decrease the number of features.

The next prompt was: “In a spark dataframe, how to output a count of the null, missing or NaN values for each column. Give a short answer.” and “What about numeric types?” I was having difficulty outputting the correct number of missing values in the missing value section of my Python notebook. I knew I needed to use select and parse through the columns so I used Claude to give a template for me to try. It responded with select statements similar to df.select([F.count(F.when(F.col(c).isNull() | F.isnan(c), c)).alias(c) for c in df.columns]). This counts the Null and NaN for both categorical and numerical columns. I adapted this for my display of the missing value counts. The exact and full output it gave is in the PDF for the chat logs (ChatLogs_Assignment2_61396.pdf).

The next prompt was: “In one column in my spark dataframe how can I transform so that each word is in its own row so I can count the main subject frequency. Give a short answer, no code.” This helped me figure out the functions to use to see the top counts for common words in the “Subject”, “Context”, “Speaker_Job”, and “Speaker” columns. It recommended using explode() and split() which I utilized and looked into further in the Spark documentation. The format I used in my code was as such: df.select(explode(split(col('subject'),',')).alias('MainSubject')).

The last prompt was: “How to use subplot() in python for side by side graphs. Give a short answer." This helped me plot side-by-side plots like in Figure 5. I knew to use the subplot() function, I just was stuck on the syntax. It gave an example for setting the subplots, specifically plt.subplot(1,2,1) and plt.subplot(1,2,2).

References

Anthropic. Claude Sonnet 4.6. https://claude.ai/. 2026

Decision Trees. scikit-learn, https://scikit-learn.org/stable/modules/tree.html. Accessed 4 Apr. 2026.

Decision Trees - RDD-based API. Apache Spark Documentation, Apache Software Foundation, https://spark.apache.org/docs/latest/mllib-decision-tree.html. Accessed 4 Apr. 2026.

Growing Decision Forests. Google Machine Learning, Google, https://developers.google.com/machine-learning/decision-forests/growing. Accessed 4 Apr. 2026.

HashingTF. Apache Flink ML Documentation, Apache Software Foundation, https://nightlies.apache.org/flink/flink-ml-docs-master/docs/operators/feature/hashingtf/. Accessed 4 Apr. 2026.

Ensemble Methods. scikit-learn, https://scikit-learn.org/stable/modules/ensemble.html. Accessed 4 Apr. 2026.

PolitiFact PolitiFact. Poynter Institute, 2026, https://www.politifact.com/.

pyspark.ml.functions.vector_to_array. Apache Spark Python API Documentation, Apache Software Foundation,

https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.functions.vector_to_array.html. Accessed 4 Apr. 2026.

tfs4. liar_dataset. GitHub, https://github.com/tfs4/liar_dataset. Accessed 4 Apr. 2026.

Wang, William Yang. "Liar, Liar Pants on Fire": A New Benchmark Dataset for Fake News Detection. Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, vol. 2, Association for Computational Linguistics, Jul. 2017, pp. 422–426, https://aclanthology.org/P17-2067/.

What Is Bagging? IBM, https://www.ibm.com/think/topics/bagging. Accessed 4 Apr. 2026.